深度解析：Codex Pet Skill

原创 lencx 浮之静 2026年5月2日 18:47

序言

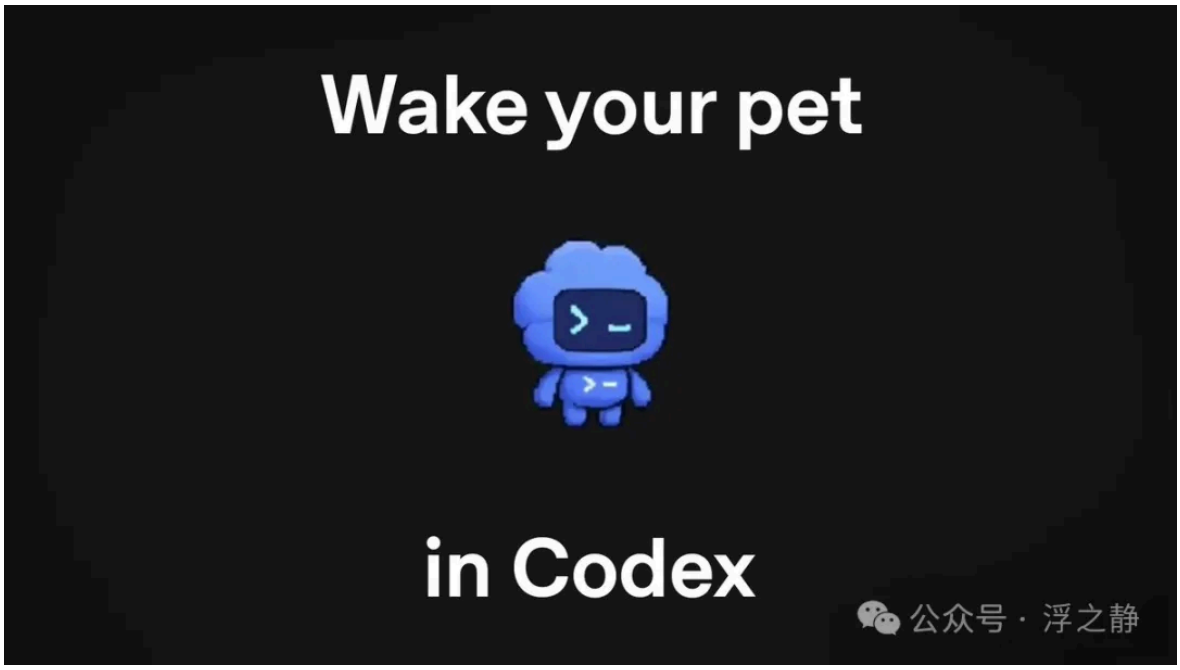
Skills 概念火了后（ *深度解析 Skills*），市面上冒出一堆看似很酷、实则很水的东西：什么“蒸馏同事”、“复刻名人”、“一键生成 PPT”、“把某某大佬装进 XXX”...，名字一个比一个唬人，截图一个比一个炸裂，实际一看，大多还是几段角色扮演 prompt，再配一点花里胡哨的 workflow 包装。它们虽然能用，但却**把 Skill 这个概念重新拉回了提示词玩具时代：用人格模仿制造幻觉，用营销标题制造稀缺感，用浅层自动化伪装成工程能力。**

很多人以为自己已经会 Skills 了，因为会写一个 `SKILL.md`，会塞几段 instructions，会让模型“像某某一样思考”，甚至会搞几个看起来很炫的 demo。但这其实还在门外打转。真正的 Skill 不是“让模型换个语气说话”，也不是“把一个 prompt 存成文件”，更不是给 Agent 套一层玄学人设。有价值的 Skill，应该把一个领域里隐性的经验、边界、工具链、失败处理、验收标准和执行流程，压缩成 Agent 可以稳定调用的**可执行协议**。

过去我也一度以为，复杂任务编排必须依赖 n8n[1]、Zapier[2] 这类强 workflow 工具：节点清晰、链路明确、触发器稳定，看起来更像“工程”。但深入研究 Skills 后，会发现另一种解法正在成型：**不是把流程全部画死在节点图里，而是把能力、约束、脚本、参考资料、验收规则和修复策略封装成可组合的执行单元**。传统 workflow 擅长确定性链路，一旦任务需要理解上下文、动态决策、局部修复、多代理协作，就很容易变成一张越来越难维护的巨型流程图。Skills 的优势恰恰在这里：它可以保持模块化，又能让 Agent 根据当前任务状态选择工具、展开步骤、调用脚本、分发子任务、回收结果，并在失败后按协议修复，而不是每次都靠人工重新接线。

所以 Codex 的 hatch-pet skill[3] 值得专门拆。它表面上只是一个“生成 Codex 电子宠物”的小功能，甚至还有点可爱、有点玩具感，但它的源码恰恰展示了什么叫真正的 Skill 工程：它不是让模型随便画一只宠物，而是把图像生成、资产协议、动画状态、子代理并行、provenance、QA、局部修复、最终打包全部串成了一条可验证的生产流水线。也正因为如此，它比那些靠标题博眼球的“人格蒸馏 Skill”更能说明问题：**Skill 的本质不是角色扮演，也不只是轻量 workflow，而是把不可控的模型能力关进可控的工程边界里。**

Hatch Pet



`hatch-pet` 表面上是 Codex 的一个“生成宠物”技能，但源码里真正有价值的不是“可爱”，而是它展示了 Codex Skills 的一种高级范式：**用 Skill 把模型生成、子代理并行、确定性脚本、QA、修复、打包、应用加载串成一个可复用工作流**。它不是 prompt 包，也不是简单的 image generation wrapper，而是一个面向 Codex app 的 **animated pet asset pipeline**。

先拆清楚两个概念：**Codex Pets** 是 Codex app 的宠物功能；官方文档说它是可选动画伙伴，可以在 `Settings → Appearance → Pets` 里选择内置宠物或刷新本地自定义宠物，也可以用 `/pet` 或命令面板唤醒/收起。这个 overlay 还会显示当前 active thread、Codex 是否运行中、等待输入或等待 review，本质上是一个轻量状态可视化层。`hatch-pet` 则是生成自定义宠物资产的 Skill，官方文档给出的安装方式是 `$skill-installer hatch-pet`，然后 Force Reload Skills，再用 `$hatch-pet create ...` 触发生成。

简介

Codex Skills 机制本身定义得很明确：一个 Skill 是包含 `SKILL.md`、可选 `scripts/`、`references/`、`assets/`、`agents/openai.yaml` 的目录；Codex 一开始只把 skill 的 name、description、路径放入上下文，只有当它决定使用这个 skill 时才读取完整 `SKILL.md`，这叫渐进式披露（progressive disclosure），用来避免技能库把上下文撑爆。

OpenAI Skills[4] 主要分为内置和手动安装：

- `.system` 技能会随新版 Codex 自动安装。
- `.curated` 和 `.experimental` 技能则通过 `$skill-installer` 安装。

`hatch-pet` 归属于 `.curated`，其目录结构是一个标准但偏高级的 Skill：

```
├── LICENSE.txt                    # Apache-2.0 许可证
├── SKILL.md                       # Skill 主入口：定义触发条件、生成
├── agents/
│   └── openai.yaml                # OpenAI/Codex interface 元数据
├── references/
│   ├── animation-rows.md          # 动画行协议：定义 8x9 atlas 中每
│   ├── codex-pet-contract.md      # Codex Pet 资产契约：atlas 尺寸
```

```
└── scripts/
    ├── prepare_pet_run.py           # 初始化 pet run：生成目录、pet_r
    ├── pet_job_status.py            # 读取 imagegen-jobs.json，输出
    ├── record_imagegen_result.py    # 摄取 $imagegen 输出：校验来源、
    ├── derive_running_left_from_running_right.py  # 条件满足时，将 r
    ├── extract_strip_frames.py      # 从 row strip 抽帧：移除 chroma
    ├── inspect_frames.py            # 检查 frame QA：尺寸、透明像素、
    ├── compose_atlas.py             # 从 frames 或已有 atlas 组装/规
    ├── validate_atlas.py            # 验证 atlas：尺寸、格式、alpha、
    ├── make_contact_sheet.py        # 生成带标签的 contact sheet，方
    ├── render_animation_videos.py   # 用 ffmpeg 从 atlas 渲染各状态动
    ├── render_animation_videos.sh   # render_animation_videos.py 的
    ├── package_custom_pet.py        # 将验证后的 spritesheet 打包为本
    ├── queue_pet_repairs.py         # 根据 qa/review.json 重新打开失
    ├── generate_pet_images.py       # 二级 fallback：当 $imagegen 不
    └── finalize_pet_run.py          # 总收口脚本：校验 job 完成与 prov
```

这意味着 `hatch-pet` 的本质不是“宠物生成器”，而是一个 **Codex-compatible animated pet packaging workflow**。它最终要产出的不是一张好看的图，而是：

```
${CODEX_HOME:-$HOME/.codex}/pets/<pet-name>/
├── pet.json
└── spritesheet.webp
```

其中 `pet.json` 描述 id、displayName、description、spritesheetPath，`spritesheet.webp` 是 Codex app 能按固定背景位置读取的精灵图。`references/codex-pet-contract.md` 明确规定 atlas 格式为 PNG/WebP，尺寸 `1536x1872`，网格 `8 columns x 9 rows`，每格 `192x208`，背景透明，未使用格必须完全透明；app 的 webview 动画通过固定行列数和 CSS background position 读取。

以下是我自己生成的宠物 Kerno `spritesheet.webp`：

使用方式

操作很简单：先在 Codex app 中执行 `$skill-installer hatch-pet` 安装这个 Skill；安装完成后，按 `Cmd + K`（macOS）或 `Ctrl + K`（Windows/Linux）打开命令菜单，搜索并执行 `Force Reload Skills`，让 Codex 重新扫描本地 Skills。刷新完成后，就可以在输入框中调用：

```
$hatch-pet create a new pet inspired by my recent projects
```

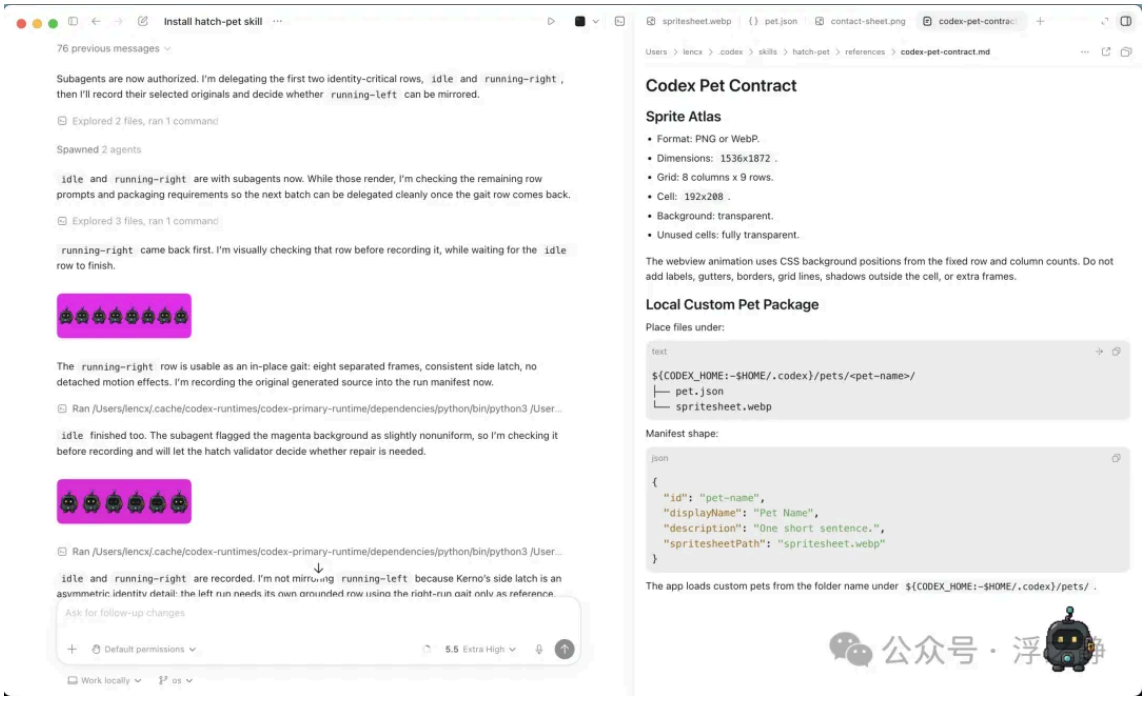
这不是普通 shell 命令意义上的 `$hatch-pet`，而是 Codex skill invocation。Codex Skills 支持显式调用和隐式调用：CLI/IDE 中可以用 `/skills` 或输入 `$` mention skill；如果任务和 skill description 匹配，Codex 也可以隐式选择技能。

如果你不给名字，`SKILL.md` 要求它从 concept、reference filename 或 personality 推断一个短名字；不给描述，则从 concept/reference 推断；不给 reference，则先通过文本生成 base pet，再把 base 作为后续所有动画行的 canonical reference。

社区并没有只把 Codex Pets 当成卖萌装饰，很多人第一时间看到了它的状态提示价值：可以把 active thread、运行中、等待输入、等待 review 这些枯燥状态，翻译成一个更自然、更低打扰的可视化反馈。这也和官方文档对 overlay 的描述对得上——宠物不是 UI 彩蛋，而是 Agent 状态的一层情绪化外壳。

📌

> 整个过程持续半个多小时，如果额度不足不建议尝试，子 agent 并行会迅速消耗额度。不自己生成也没关系，可以通过 `https://github.com/lencx/ai-pet` 来获取我生成的宠物（目前只上传了一个，会陆续新增，也欢迎大家来贡献自己宠物）。



资产协议

> 为什么是 8x9 atlas，而不是随便一堆 GIF?

`references/animation-rows.md` 是这个 Skill 的资产契约核心。它规定 Codex app 读取一个固定 atlas：8 列 9 行、每格 `192x208`。9 行分别是：

```
0 idle           6 frames
1 running-right  8 frames
2 running-left   8 frames
3 waving         4 frames
4 jumping        5 frames
5 failed         8 frames
6 waiting        6 frames
7 running        6 frames
8 review         6 frames
```

未使用的 cell 必须完全透明。每行还带有语义：`idle` 是呼吸/眨眼，`running-right` 是向右跑，`failed` 是失败/沮丧反应，`review` 是专注检查状态。

这点很关键。它说明 `hatch-pet` 不是在生成“角色形象”，而是在生成一个**状态机可消费的动画表**。Codex app 需要把 agent state 映射成宠物动作，所以每一行动画都要有稳定语义。比如：

```
agent idle      -> idle row
agent running   -> running / running-right / running-left row
agent waiting   -> waiting row
agent failed    -> failed row
agent reviewing -> review row
```

这就是为什么源码里反复强调：不能接受“看起来不错但状态不清”的图。一个漂亮的宠物插画不等于可用的 pet asset。对于 app 来说，可用性来自固定尺寸、固定行列、透明背景、动作语义、循环完整度和身份一致性。

SKILL.md 核心设计

> 把不可控图像生成关进工程边界

`SKILL.md` 最值得看的部分是 Generation Delegation。它规定所有正常视觉生成都要委托给 `$imagegen`，不能直接调用 Image API；本 skill 自己的脚本只做确定性工作：准备 prompts/manifests、摄取 imagegen 输出、抽帧、验证、拼 atlas、生成 QA media、打包。

这句话背后有个很强的架构边界：

- **非确定性部分**：$imagegen 负责
- **确定性部分**：hatch-pet scripts 负责

也就是说，模型可以生成视觉内容，但不能自己“假装完成生产”。`SKILL.md` 明确禁止用本地 Python/Pillow、SVG、canvas、HTML/CSS 或脚本去画、拼、变形、伪造宠物视觉；普通流程最多 10 个视觉生成任务：1 个 base pet + 9 个 row strip。唯一特例是 `running-left`，可以在 `running-right` 已生成、已视觉检查、并确认适合镜像后由脚本水平镜像得到。

这就是为何说它不是 prompt 包的原因。它内置了**反作弊/反幻觉边界**：

- 不能手改 imagegen-jobs.json 标记完成
- 不能复制 run/tmp/decoded 中的本地文件冒充生成结果
- 不能写 helper script 填充 row outputs
- 必须用 record_imagegen_result.py 记录真实 $imagegen 输出
- row job 必须带 grounding images
- base job 才允许 prompt-only

从 agent 工程角度看，这是一种非常成熟的 skill 写法：它不相信模型“说完成了”，只相信文件、hash、manifest、QA 结果和可检查产物。

Prompt 策略

> 不是“更好看”，而是“更像 Codex digital pet”

`hatch-pet` 的美术风格也写得非常具体。默认风格是 Codex app 内置 digital pets：小型 pixel-art-adjacent mascot，compact chibi proportions，chunky silhouette，1–2px 深色描边，stepped/pixel edges，有限色板，flat cel shading，简单表情，小四肢。它明确禁止 polished illustration、painterly rendering、anime key art、3D render、glossy app icon、realistic fur、soft gradients、高细节抗锯齿等。

这段不是审美偏好，而是工程约束。因为最终 cell 只有 `192x208`，高细节会在小尺寸下糊成噪声；柔光、阴影、半透明效果会破坏 chroma key / alpha extraction；复杂饰品会导致行间身份漂移。

它还对透明背景和 effects 做了极细规则：允许的效果必须和宠物轮廓接触或重叠，必须在同一 frame slot 内，必须 opaque、hard-edged、pixel-style，且不能用 chroma-key 邻近色；禁止 wave marks、motion arcs、speed lines、afterimages、blur、detached stars、floating punctuation、shadows、glows、speech bubbles、UI panels、checkerboard transparency 等。

这说明 `hatch-pet` 的 prompt 不是“增强生成质量”，而是在压缩生成空间：减少图像模型最爱犯的花活，把输出限制到 sprite production contract 里。

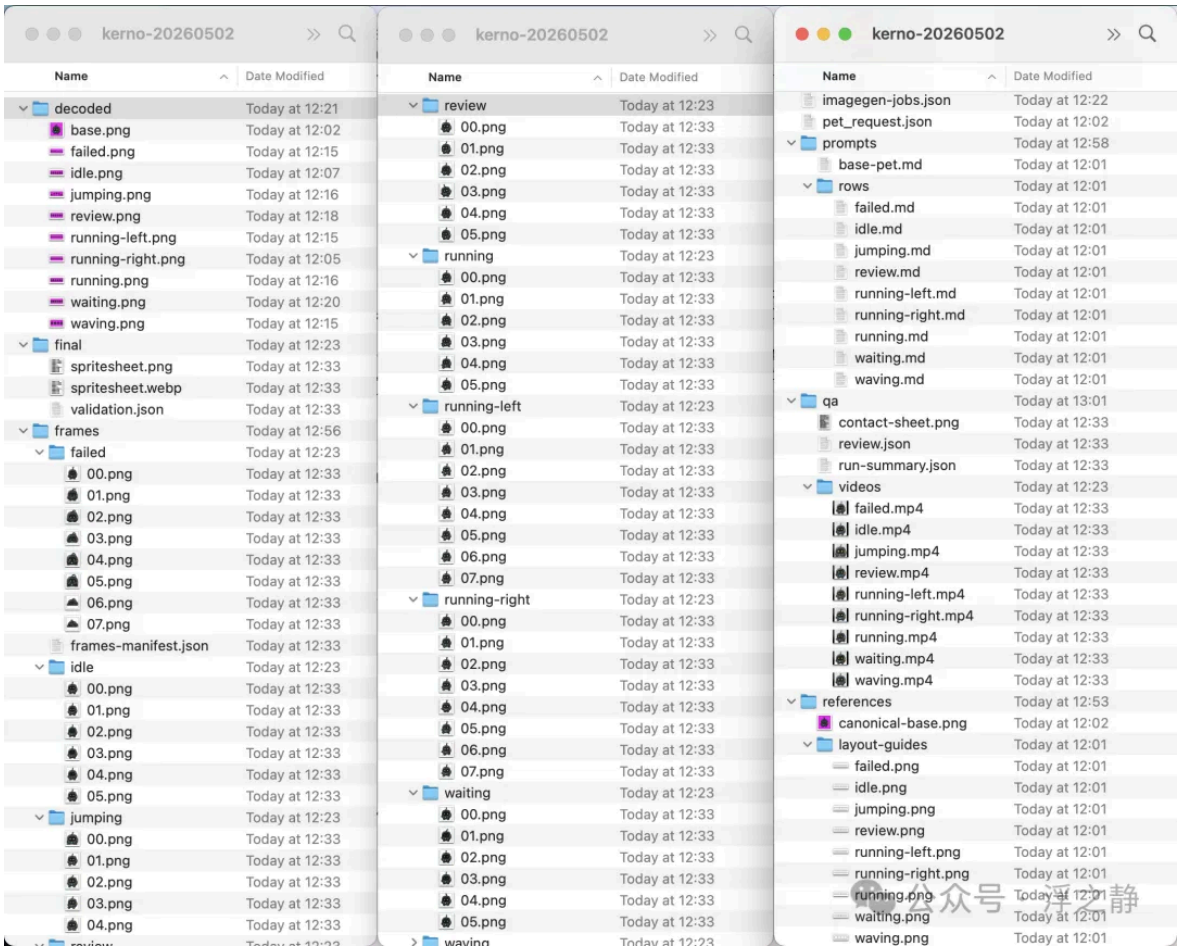
编排起点

`prepare_pet_run.py` 的职责是创建一次 pet run 的工作目录、prompt 文件和 `imagegen-jobs.json`。它内置 atlas 参数：8 列 9 行、cell `192x208`，并定义全部动画行：`idle`、`running-right`、`running-left`、`waving`、`jumping`、`failed`、`waiting`、`running`、`review`。

它会做几件关键事：

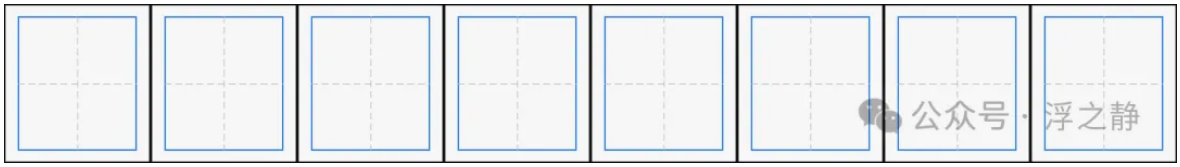
1. 推断 pet name / pet id / description / pet notes
2. 复制 reference images 到 run/references
3. 自动选择 chroma key
4. 生成 layout guide images
5. 写 pet_request.json
6. 写 prompts/base-pet.md
7. 写 prompts/rows/.md
8. 写 imagegen-jobs.json

以下是宠物 Kerno 生成的中间产物，macOS 中位于以下路径 `/private/tmp/hatch-pet/kerno-20260502`：



最有意思的是 chroma key 选择。脚本不是固定用绿色，而是从候选色中选择和参考图像像素距离较远的颜色，候选包括 magenta、cyan、yellow、blue、orange、green。也就是说，它提前规避“宠物本体颜色被背景抠掉”的问题。`SKILL.md` 也明确要求使用 `pet_request.json` 中存储的 chroma key，不要强行固定 green screen。

另一个关键点是 layout guide。每个 row job 都会带一个对应状态的 layout guide，用作 frame count、spacing、centering、安全边距的视觉参考；但 prompt 又要求不能把 guide 的框线、标记、颜色复制到最终图像里。以失败态为例，大概长这样：



这其实是在解决图像模型生成 sprite strip 的经典问题：模型很容易把 6 帧画成 5 帧半、间距乱、角色跨格、边缘裁切。layout guide 是给模型看的“隐形施工图”，不是最终资产的一部分。

用 manifest 约束生成

`prepare_pet_run.py` 生成的 job manifest（`imagegen-jobs.json`）是这个 Skill 的控制中心。它先创建一个 `base` job，然后为每个 state 创建 row-strip job。base job 可以在没有 reference 时 prompt-only；但 row-strip job 都要求 grounded generation，必须附带 canonical base、decoded/base、layout guide，以及原始 reference。

`running-left` 是特殊 job：它依赖 `base` 和 `running-right`，并带有 mirror policy。只有当 `running-right` 视觉上足够对称，镜像不会破坏身份、道具方向、左右手、标记、文字、光照和方向语义时，才允许由 `derive_running_left_from_running_right.py` 镜像生成；否则必须作为普通 row 重新通过 `$imagegen` 生成。

这里非常工程化。它不是让模型在脑子里“记住流程”，而是把流程固化成 manifest：

```
{
  "id": "running-left",
  "kind": "row-strip",
  "depends_on": ["base","running-right"],
  "generation_skill": "$imagegen",
  "requires_grounded_generation": true,
  "allow_prompt_only_generation": false,
  "mirror_policy": {
    "may_derive_from": "running-right",
    "derivation": "horizontal-mirror",
    "requires_explicit_approval": true
  }
}
```

这就是 Harness 思维（*深度解析：Harness Engineering*）：模型只负责选择下一步和执行局部任务，状态、依赖、边界、验收由外部结构保存。

📌 imagegen-jobs.json

这份 JSON 本质上是一次 `hatch-pet` 生成任务的 **Job Manifest**，也就是运行清单。顶层字段负责描述本次 run 的基本信息：`schema_version` 表示结构版本，`created_at` 表示创建时间，`run_dir` 指向工作目录，`primary_generation_skill` 记录主要生成能力，例如 `$imagegen`。真正的核心是 `jobs` 数组：每个 job 都是一个可独立追踪的任务单元，例如 `base`、`idle`、`running-right`、`jumping`、`review`。这些 job 通过 `prompt_file`、`input_images`、`output_path`、`depends_on`、`parallelizable_after` 等字段，描述自己使用什么 prompt、依赖哪些参考图、产物写到哪里、必须等待哪些任务完成，以及何时可以并行执行。

以下是结构化后 JSON，方便展示此 manifest 设计：

```
{
  "schema_version": 1,
  "created_at": "ISO-8601 timestamp",
  "run_dir": "absolute-or-relative-run-directory",
  "primary_generation_skill": "skill-or-tool-name",

  "jobs": [
```

```
    {
      "id": "job-id",
      "kind": "job-type",
      "status": "pending | complete",

      "prompt_file": "path/to/prompt.md",
      "input_images": [
        {
          "path": "path/to/input.png",
          "role": "why this input is used"
        }
      ],

      "output_path": "path/to/output.png",
      "depends_on": ["dependency-job-id"],
      "parallelizable_after": ["dependency-job-id"],
```

这套结构的重点不只是“记录结果”，而是把生成过程变成一张可编排、可验证、可修复的任务图。`generation_skill`、`requires_grounded_generation`、`allow_prompt_only_generation`、`identity_reference_paths` 用来约束生成方式，避免后续动画行只靠 prompt 发散；`source_path`、`source_provenance`、`source_sha256`、`output_sha256`、`completed_at` 和 `metadata` 用来记录来源与校验信息，保证产物可追溯；`mirror_policy` 则表达特殊派生规则，比如 `running-left` 是否可以从 `running-right` 镜像得到。需要注意的是，manifest 中持久化的 job `status` 主要是 `pending` 和 `complete`；`blocked` 是 `pet_job_status.py` 根据依赖缺失派生出来的视图概念，`failed` 则通常体现在 QA/review 结果或 repair 流程里，而不是常规 manifest status。最后的 `canonical_identity_reference` 是全局身份锚点，通常来自 `base` job，用来保证所有动画行仍然是同一只宠物。

子代理设计

> 并行生成，但父代理独占写权限

`hatch-pet` 要求在 base job 记录完成、`references/canonical-base.png` 存在后，row-strip visual generation 必须使用 subagents，除非用户明确说不要。父代理负责准备 run、生成并记录 base、查看 job status、先派发 `idle` 和 `running-right`，再决定 `running-left` 是否镜像，之后派发剩余 row jobs。

这和 Codex Subagents 的官方能力对得上：Codex 可以生成 specialized agents 并行执行任务，然后收集结果；这种模式适合复杂且高度并行的任务，但会消耗更多 tokens。官方文档也说，Codex 负责 spawn、routing、waiting、closing agent threads，并且 subagents 继承当前 sandbox policy。

`hatch-pet` 的关键不是“用了子代理”，而是它定义了非常严格的写边界：

```
Subagent 可以:
- 读取 row prompt
- 调用 $imagegen
```

```
- 检查候选图
- 返回 selected_source 和 qa_note

Subagent 不可以:
- 修改 imagegen-jobs.json
- 拷贝文件进 decoded/
- 执行 record_imagegen_result.py
- 镜像 running-left
- finalize
- repair
- package
```

父代理独占 manifest 和 package 写入。这避免了并行写冲突、来源混乱和 provenance 污染。对于 agent 系统来说，这是非常值得借鉴的模式：**并行任务可以分发，状态提交必须集中。**

资产溯源

> 从“生成了图”到“可追溯资产”

图像生成完成后，不能直接把图片丢进最终目录。必须运行：

```
python "$SKILL_DIR/scripts/record_imagegen_result.py" \
  --run-dir /absolute/path/to/run \
  --job-id <job-id> \
  --source /absolute/path/to/generated-output.png
```

这个脚本会检查 job 依赖是否完成、要求的 grounding images 是否存在，然后把 source copy 到 job 的 `output_path`，记录 `source_path`、`source_provenance`、`source_sha256`、`output_sha256`、metadata、completed_at。base job 还会额外复制出 `references/canonical-base.png`，并写入 `pet_request.json` 和 manifest。

这一步非常重要。它把“模型给了我一张图”转换为“这张图是哪个 job 的产物、从哪里来、hash 是什么、是否满足依赖”的可审计记录。

`finalize_pet_run.py` 后续还会验证这些 hash。如果 job 缺少 `source_sha256`，它会报错并提示必须用 `record_imagegen_result.py`，而不是手改 `imagegen-jobs.json`。如果 decoded output 和 source hash 不一致，也会报错，提示不要本地改写 decoded visual outputs。

这就是 anti-hallucination 的硬约束：模型不能靠“我已经处理好了”过关，必须留下可验证的 provenance。

打包链路

> 从 row strips 到最终宠物包

当所有 imagegen jobs 都 complete 后，最终执行：

```
python "$SKILL_DIR/scripts/finalize_pet_run.py" \
  --run-dir /absolute/path/to/run
```

`finalize_pet_run.py` 的流程大致是：

1. 读取 pet_request.json
2. 检查 imagegen-jobs.json 中所有 job 都已 complete
3. 验证每个 job 的 source provenance 和 hash
4. extract_strip_frames.py 抽帧
5. inspect_frames.py 检查每帧
6. compose_atlas.py 拼 spritesheet.png / spritesheet.webp
7. validate_atlas.py 验证最终 atlas
8. make_contact_sheet.py 生成 QA contact sheet
9. render_animation_videos.py 生成 preview videos
10. package_custom_pet.py 打包到 ${CODEX_HOME:-$HOME/.codex}/pets/，或通过 --package-dir 指定目录
11. 写 qa/run-summary.json

源码里可以看到，finalize 会先 `require_complete_jobs`，不完整就提示用 `pet_job_status.py` 继续完成；然后执行 `extract_strip_frames.py`、`inspect_frames.py`，如果 review 失败，会输出 `repair_hint`：运行 `queue_pet_repairs.py`，重新生成 reopened row jobs，再 finalize。通过后才 compose atlas、validate、生成 contact sheet、videos，并 package。

这是一条完整的生产链，而不是单轮生成。它天然支持失败、修复、重跑、验收。

QA 策略

> 确定性 QA 只检查“结构”，不检查“好不好看”

`validate_atlas.py` 检查的是最终 spritesheet 的工程正确性：尺寸必须是 `1536x1872`，格式必须是 PNG 或 WebP，除非显式 allow，否则必须有 alpha channel；它逐个 cell 统计 alpha 非零像素，used cell 太稀疏会报错，unused cell 非透明会报错，used cell 近似全 opaque 会被视为背景没抠干净。

但 `SKILL.md` 反复强调：deterministic validation necessary but not sufficient。即使 `qa/review.json` 和 `final/validation.json` 没错误，也必须人工/视觉检查 contact sheet，确认没有 identity drift：不能换物种、换体型、换脸、换标记、换调色板、换道具设计、换轮廓。

这点非常专业。自动 QA 可以证明尺寸、透明、帧数、空格、alpha；但它不能证明“这还是同一只宠物”。所以 `hatch-pet` 把验收拆成两层：

- **结构正确性**：脚本检查
- **视觉一致性**：模型/人类视觉检查

这和真实生产系统很像：schema validation 只能证明数据形状正确，不能证明语义正确。

Repair Workflow

> 不是重来，而是最小失败范围修复

如果 finalization 因 row QA 失败而停止，`SKILL.md` 要求运行：

```
python "$SKILL_DIR/scripts/queue_pet_repairs.py" \
  --run-dir /absolute/path/to/run
```

然后只重新生成 reopened row job。它明确要求 regenerate the smallest failing scope：失败的是某一行，就修这一行，不要重做整张 sheet。对于 identity repairs，要用 canonical base、original references、contact sheet、exact row failure note 作为 grounding context。

`qa-rubric.md` 也强调 repair policy：原则上先定位最小失败范围，只有 identity 或 layout 广泛损坏时才 full atlas regeneration；而当前自动化生产路径里，`queue_pet_repairs.py` 会按失败 row 重新打开整行动画 job，并在 repair note 中要求重生成整行，而不是只修某一帧。

这又是一个高级点：它不是“生成失败就重新 prompt 一遍”，而是把失败局部化。对图像生成这种高成本、不稳定任务来说，局部修复比整体重跑更可控。

Secondary fallback

> 有 API fallback，但不是默认路径

`scripts/generate_pet_images.py` 是 secondary fallback。`SKILL.md` 写得很清楚：只有当已安装的 `$imagegen` system skill 不可用或当前环境不能调用时，才使用这个 fallback；普通宠物创建应该委托给 `$imagegen`，因为 `$imagegen` 拥有 built-in-first image generation policy 和 CLI fallback 行为。fallback 需要 `OPENAI_API_KEY`，并可用 `--model gpt-image-2 --states all` 执行。

这体现了另一个边界：`hatch-pet` 不想自己接管所有图像生成策略。它只定义宠物生产协议；视觉生成策略交给系统级 `$imagegen`。这是一种 skill composition：

```
hatch-pet   = domain workflow
$imagegen   = visual generation capability
scripts     = deterministic asset compiler
```

这比把所有东西塞进一个巨型脚本更干净。

Skills 高级范式

`hatch-pet` 最值得学习的不是“怎么做宠物”，而是它展示了一个成熟 Skill 应该如何写。

- **description 是路由入口**。Codex 初始上下文里只有 skill name、description、path，所以 description 必须精准说明何时触发。`hatch-pet` 的 description 一开始就写：create、repair、validate、preview、package Codex-compatible animated pet spritesheets，并列出 8x9 atlas、transparent unused cells、row-by-row animation prompts、QA contact sheets、preview videos、pet.json packaging。这个 description 不是营销文案，是 router spec。
- **SKILL.md 是操作系统级 workflow contract**。它规定谁能生成、谁能写 manifest、什么时候用 subagents、什么时候必须停止、哪些行为禁止、什么算验收。它不是“建议模型这样做”，而是在给 agent 建立行为边界。
- **scripts 是确定性编译器**。视觉内容来自模型，但资产结构不能靠模型猜。抽帧、拼 atlas、透明验证、hash、package 都由脚本完成。这种组合非常适合 Agent：模型处理语义，脚本处

理确定性。

- **references 是协议文档**。`animation-rows.md` 规定动画状态，`codex-pet-contract.md` 规定 app 消费格式，`qa-rubric.md` 规定验收标准。它们不是给用户看的说明书，而是给 agent 和脚本共享的生产契约。
- **subagents 并行但不共享写权限**。这点尤其值得用于自己的 AgentOS 架构：并行子代理可以做探索、生成、候选筛选，但统一状态、审计、manifest、package 必须由父控制面写入。

局限性

`hatch-pet` 的源码也暴露出几个天然限制。

它依赖 `$imagegen` 的视觉稳定性。Skill 通过 base canonical reference、layout guide、row prompt、QA 修复降低漂移，但无法从根上保证所有动画行完全一致。所以它才要求 contact sheet 视觉检查，并把 identity drift 视为 blocker。

它生成的是固定格式资产，不是任意动画系统。8x9、192x208、固定 row state 是 Codex app 的消费协议。你可以把这个思路移植到别的 app，但不能假设 `spritesheet.webp` 直接适配任意 UI。

它的 QA 仍然有语义空洞。`validate_atlas.py` 能检查 alpha、尺寸、空 cell、近似 opaque，但不能判断“review 状态是否真的像 review”、“failed 是否太夸张”、“running-left 镜像是否破坏角色设定”。这部分仍然依赖视觉检查和模型判断。

它的子代理要求会增加成本。官方 Subagents 文档也说明，子代理各自执行模型和工具工作，会比单代理消耗更多 tokens。`hatch-pet` 选择子代理，是因为 9 行动画天然并行；但如果只是小修一行，顺序执行可能更便宜。

Agent 工程启发

如果只把 `hatch-pet` 看成“一个会画宠物的 Skill”，就会低估它真正展示的东西。它更像一个缩小版 Agent 生产系统：把用户一句模糊的创意请求，转换成一条有输入规范、任务图、来源证明、确定性编译、QA、修复和最终打包的资产流水线。

> 真正的 Skill 不是“提示词模板”，而是把一个领域的隐性经验、失败边界和产物协议，编译成 Agent 可以执行、可以恢复、可以验收的生产流程。

这也是 `hatch-pet` 和普通 prompt wrapper 的根本差别。普通 prompt wrapper 往往只是告诉模型“你要怎么想、怎么写、怎么生成”；`hatch-pet` 则先定义产物是什么，再定义生成过程如何受控。它不满足于“画一只好看的宠物”，而是要求最后产出一个 Codex app 能加载的 animated pet package。美术只是其中一个环节，真正的目标是一个符合协议的 artifact。

从这个角度看，`hatch-pet` 的核心链路可以抽象成：

```
intent
-> request
-> job manifest
-> generated candidates
-> recorded provenance
-> compiled artifact
-> QA result
```

```
-> targeted repair
-> packaged asset
```

这条链路给 Agent 工程的几层启发：

- **不要把模型输出直接当成果，而要把它当生产材料**。模型可以生成 base pet，可以生成 row strip，可以判断某个候选是否更好；但这些输出都只是材料。只有经过 `record_imagegen_result.py` 记录来源、经过脚本抽帧和拼 atlas、经过 `validate_atlas.py` 和 contact sheet 检查、再经过 `package_custom_pet.py` 打包之后，它才变成最终资产。换句话说，模型是 creative worker，不是 trusted committer。
- **关键状态不能藏在上下文里，必须外部化成 manifest**。长任务里的对话上下文会压缩、会遗忘、会被新的推理覆盖；`imagegen-jobs.json` 不会。它把 job 的依赖、输入、输出、来源、派生规则、记录者和完成状态写成可检查结构，让任务可以恢复、可以 diff、可以被另一个脚本重新验证。对 AgentOS 来说，这比“让模型记住自己做到哪一步”可靠得多。凡是需要跨步骤、跨代理、跨失败恢复的任务，都应该有类似的外部状态机，而不是只依赖 prompt 里的计划。
- **多代理的关键不是并行，而是提交权隔离**。`hatch-pet` 让 subagents 生成动画行，是因为 row strips 天然可以并行；但它更重要的设计是写边界。子代理可以读 prompt、调用 `$imagegen`、挑选 source、写 QA note，却不能改 manifest、不能 copy 到 decoded、不能 record、不能 mirror、不能 finalize、不能 package。父代理才是 control plane，子代理只是 worker plane。这个模式值得抽象成一条规则：并行任务可以分发，truth commit 必须集中。
- **QA 要承认结构正确和语义正确不是一回事**。脚本能检查尺寸、alpha、空 cell、帧数、hash、透明背景，但不能证明“这还是同一只宠物”，也不能证明 review 状态真的像 review。`hatch-pet` 没有假装脚本能解决一切，而是把自动 QA 和视觉检查拆开。这个分层非常重要。很多 Agent 系统只做 schema validation，然后误以为任务完成；但真实生产里，schema correct 只是底线，semantic correct 才是用户真正关心的结果。
- **repair 比 retry 更像工程**。失败后重新生成一整套资产当然简单，但成本高、漂移大，也容易把原本好的部分重新破坏。`hatch-pet` 的方式是定位失败 row，归档旧输出，追加 repair note，重开对应 job，再重新 finalize。它承认模型生成有波动，但用流程把波动限制在最小失败范围内。这个思想可以迁移到代码、UI、文档、测试、知识库等任务：成熟 Agent 不应该只会 retry，而应该知道失败在哪里、修复边界在哪里、如何保留已经通过验收的部分。

所以，对 AgentOS / control plane 方向来说，`hatch-pet` 的启发不是“给 Agent 加一个宠物功能”，而是展示了一种更普适的构造方法：

```
让模型生成候选
让 manifest 保存状态
让脚本编译产物
让 QA 切分结构与语义
让 repair 局部收敛
让 control plane 统一提交
```

这比单纯给 Agent 更多 prompt 更重要。prompt 可以提升一次回答的质量，但协议、manifest、provenance、编译器和 QA 才能提升一个系统的稳定性。`hatch-pet` 做的是动画宠物，但它真正示范的是如何把不可控的模型能力，接进一个可控的产物系统。

这也解释了为什么它和 vibe coding 并不矛盾。很多人把 vibe coding 理解成少约束、快生成、跟着感觉走；但 `hatch-pet` 展示的是更成熟的版本：创意层开放，协议层收紧，提交层验证，失

败层可修。只有底层边界足够硬，上层创意才敢流动。否则所谓 vibe 只是一次灵感截图；有了产物协议和验收链路，它才会变成一个真的能被应用加载、复用和长期维护的 artifact。

Codex /goal 命令

Codex v0.128.0 是一次重要更新，核心变化是引入了更强的目标驱动执行机制。新增的 `/goal <objective>` 命令允许用户为 Codex 设置明确目标；当 agent 完成一轮执行后，如果用户没有继续输入，Codex 会自动注入提示，引导模型选择下一个具体动作继续推进。这个机制会将 goal 的需求映射到可验证证据上，例如文件变更、测试结果、PR 状态等；模型只能通过更新 goal 来标记任务完成。因此，它更像是一个内置的 Ralph loop++：不再停留于单轮响应，而是围绕目标持续执行、检查证据并推进任务，直到目标完成或 token 预算耗尽。goal 命令需要在配置中开启，默认不会显示，config.toml [features] 中新增 goals = true。

在源码中也有一些证据 codex-rs/goals[5]，感兴趣的朋友自行了解。此命令一出，终于不用再给 codex 连续发送“继续”了...

References

[1] n8n: *https://n8n.io*

[2] Zapier: *https://zapier.com*

[3] hatch-pet skill: *https://github.com/openai/skills/tree/main/skills/.curated/hatch-pet*

[4] OpenAI Skills: *https://github.com/openai/skills*

[5] codex-rs/goals: *https://github.com/openai/codex/blob/rust-v0.128.0/codex-rs/core/templates/goals*

喜欢作者

AI 系列 · 目录

上一篇 · Claude Opus 4.7 & ChatGPT Image

修改于2026年5月2日